

파머완 8장 Part2(8.6~8.9)

8.6 토픽 모델링(Topic Modeling) - 20 뉴스그룹



토픽 모델링 - 문서 집합에 숨어 있는 주제를 찾아내기

※ 주로 사용되는 기법

- LSA(Latent Semantic Analysis)
- LDA (Latent Dirichlet Allocation)

```
from sklearn.decomposition import LatentDirichletAllocation
```



사이킷런의 LatentDirichletAllocation 클래스

※ 파라미터

- n_components : 토픽의 개수
- random_state : 예제를 수행할 때마다 결과가 똑같게 하기 위해 입력.

※ 객체의 속성

- components_ :
 - 개별 토픽별로 각 word 피처가 얼마나 많이 그 토픽에 할당됐는지 수치로 표현
 - 높은 값일 수록 해당 피처는 그 토픽의 중심 word가 됨.

8.7 문서 군집화 소개와 실습(Opinion Review)



비슷한 텍스트 구성의 문서를 군집화(clustering)

동일한 군집에 속하는 문서를 같은 카테고리 소속으로 분류 → 텍스트 분류(지도 학습)와 유사

비지도학습

8.8 문서 유사도



일반적으로 코사인 유사도(Cosine Similarity) 사용

벡터들의 유사도를 비교할 때 벡터의 크기보다는 벡터의 상호 방향성이 얼마나 유사한지 비교

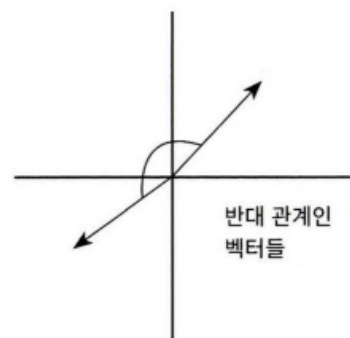
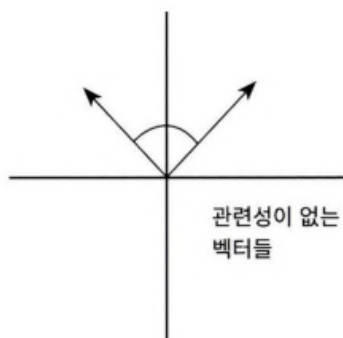
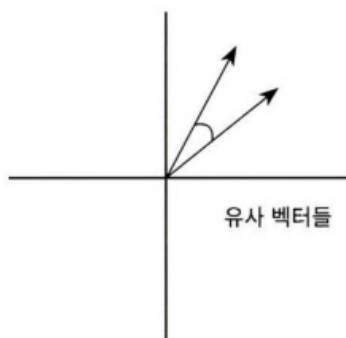
>> 벡터 사이의 사잇각



두 벡터 사잇각



두 벡터들의 사잇각 → 벡터의 상호관계



벡터들의 유사도



A와 B의 내적 = A크기 * B크기 * 코사인

$$A * B = \|A\| \|B\| \cos \theta$$

$$\text{similarity} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

코사인 유사도가 문서의 유사도 비교에 제일 많이 사용되는 이유

- 문서를 피쳐 벡터화하면 차원 多 희소행렬이 됨
→ 희소 행렬에서 문서/문서벡터 간의 크기에 기반한 유사도 지표(ex.유클리드 거리)는 정확도가 떨어짐.

예)

A - 머신러닝 5번 언급, B - 머신러닝 3번 언급

but A 문서가 B 문서보다 10배 이상 크다면 → B가 더 머신러닝과 관련된 문서!

사이킷런의 코사인 유사도 측정 API

```
from sklearn.metrics.pairwise import cosine_similarity()
```

- 희소행렬, 밀집행렬, 행렬, 배열, 다 가능 → 별도의 변환 필요 ❌

8.9 한글 텍스트 처리 - 네이버 영화 평점 감성 분석



대표적인 파이썬 기반의 한글 형태소 패키지 - KoNLPy

형태소 : 단어로써 의미를 가지는 최소단위

- 형태소 분석 (Morphological analysis)
: 말뭉치를 형태소 어근 단위로 쪼개고 각 형태소에 POS 부착

+ 한글 NLP 처리의 어려움

- 띄어쓰기 + 다양한 조사
- 띄어쓰기 > 위치에 따라 의미 왜곡, 고등교육을 받아도 틀릴 수 있음
- 조사 > 경우의 수가 많으면 어근 추출(Stemming/Lemmatization) 등의 전처리시 제거 힘들

+ KoNLPy

- 이전에는 대부분 C/C++ 또는 Java 기반 패키지로 개발, KoNLPy는 java→python wrapper class
- 5개의 형태소 분석 모듈 모두 사용 가능 : 꼬꼬마(Kkma), 한나눔(Hannanum), Komoran, 은전한닢(Mecab) 프로젝트, Twitter
- Java(1.7이상, Os 비트 수 일치), JPyype1(32/64 맞춰 설치) 우선 설치 필요
- cmd에 pip install konlpy / anaconda에 conda install -c conda-forge jpyype
- morphs() : 입력인자로 들어온 문장을 형태소 단어형태로 토큰화, list 객체로 반환